Dynamic Programming

Paradigms

A **greedy algorithm** always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.

A **divide-and-conquer** algorithm breaks the problem into subproblems; solves them; and combines the solutions to obtain a solution to the original problem.

A **dynamic programming** algorithm breaks the problem into a series of overlapping subproblems; combine solutions of smaller subproblems to obtain a solution to the original problem.

Matrix-chain Multiplication Problem

Input: Given a sequence (chain) $\langle A_1, A_2, \ldots, A_n \rangle$ of n matrices,

Question: compute the product $A_1 A_2 \ldots A_n$ using standard matrix multiplication (not Strassen's method) while minimizing the number of scalar multiplications.

Matrix-chain Multiplication Problem

Input: Given a sequence (chain) $<A_1, A_2, \ldots, A_n>$ of n matrices,

Question: compute the product $A_1 A_2 \ldots A_n$ using standard matrix multiplication (not Strassen's method) while minimizing the number of scalar multiplications.

Example: A1: 10 X 100

A2: 100 X 5

A3: 5 X 50

Compute A1A2A3

Matrix-chain Multiplication Problem

Input: Given a sequence (chain) $<A_1, A_2, \ldots, A_n>$ of n matrices,

Question: compute the product $A_1 A_2 \ldots A_n$ using standard matrix multiplication (not Strassen's method) while minimizing the number of scalar multiplications.

Example: A1: 10 X 100

A2: 100 X 5

A3: 5 X 50

Compute A1A2A3

((A1A2)A3) (A1(A2A3))

Matrix-chain Multiplication Problem

Input: Given a sequence (chain) $<A_1, A_2, \ldots, A_n>$ of n matrices,

Question: compute the product $A_1 A_2 \ldots A_n$ using standard matrix multiplication (not Strassen's method) while minimizing the number of scalar multiplications.

Example: A1: 10 X 100

A2: 100 X 5

A3: 5 X 50

Compute A1A2A3

How to parenthesize the matrices?

((A1A2)A3) (A1(A2A3))

Not actually multiplying matrices; **deciding an order with the lowest cost**.

Counting the number of parenthesizations

Not actually multiplying matrices; **deciding an order with the lowest cost**.

P(n)= #ways to parenthesize a product of n matrices.
P(1)= 1.

Counting the number of parenthesizations

Not actually multiplying matrices; **deciding an order with the lowest cost**.

P(n)= #ways to parenthesize a product of n matrices.
P(1)= 1.

$$P(n) = \begin{cases} 1 & \text{if } n = 1 , \\ \sum_{k=1}^{n-1} P(k)\,P(n-k) & \text{if } n \geq 2 . \end{cases}$$

Counting the number of parenthesizations

Not actually multiplying matrices; **deciding an order with the lowest cost**.

P(n)= #ways to parenthesize a product of n matrices.
P(1)= 1.

$$P(n) = \begin{cases} 1 & \text{if } n = 1 , \\ \sum_{k=1}^{n-1} P(k) P(n-k) & \text{if } n \geq 2 . \end{cases}$$

$P(n)=\Omega(4^n / n^{3/2})$

Catalan number

Lower bound

Guess, $P(n)=\Omega(2^n)$

$$P(n) = \begin{cases} 1 & \text{if } n = 1 , \\ \sum_{k=1}^{n-1} P(k) P(n-k) & \text{if } n \geq 2 . \end{cases}$$

Using substitution method

Multiplication of Matrices

```
MATRIX-MULTIPLY(A, B)
1  if A.columns ≠ B.rows
2       error "incompatible dimensions"
3  else let C be a new A.rows × B.columns matrix
4       for i = 1 to A.rows
5            for j = 1 to B.columns
6                 c_ij = 0
7                 for k = 1 to A.columns
8                      c_ij = c_ij + a_ik · b_kj
9       return C
```

Structure of an optimal solution

Let A[i:j] be the matrix product $A_i A_{i+1} \ldots A_j$

Structure of an optimal solution

Let A[i:j] be the matrix product $A_i A_{i+1} \ldots A_j$

To optimally parenthesize A[i:j], we must split between A_k and A_{k+1} for some $i \leq k < j$.

The way we parenthesize A[i:k] within the optimal parenthesization A[i:j] should be optimal. Why?

Otherwise,

Structure of an optimal solution

Let A[i:j] be the matrix product $A_i A_{i+1} \ldots A_j$

To optimally parenthesize A[i:j], we must split between A_k and A_{k+1} for some $i \leq k < j$.

The way we parenthesize A[i:k] within the optimal parenthesization A[i:j] should be optimal. Why?

Otherwise, if there's a less costly way to parenthesize it, you'd use it and get a parenthesization of A[i:j] with a lower cost.

Structure of an optimal solution

Suppose that optimal parenthesization of A[i:j] splits between A_k and $A_{k+1,}$ for some $i \leq k \leq j-1$

Then the parenthesization of A[i:k] must be optimal.

Otherwise, if there's a less costly way to parenthesize it, you'd use it and get a parenthesization of A[i:j] with a lower cost.

Same for A[k+1:j].

Need to consider all possible splits.

Structure of an optimal solution

Let A[i:j] be the matrix product $A_i A_{i+1} \ldots A_j$

If $i < j$, then must split between A_k and A_{k+1} for some $i \leq k < j$)
Then compute A[i:k] and A[k+1:j], and multiply them together.

Cost is

Structure of an optimal solution

Let A[i:j] be the matrix product $A_i A_{i+1} \ldots A_j$

If $i < j$, then must split between A_k and A_{k+1} for some $i \leq k < j$)
Then compute A[i:k] and A[k+1:j], and multiply them together.

Cost is

cost of computing A[i:k]
+ cost of computing A[k+1:j]
+ cost of multiplying them together.

Example

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25

$m[2, 5] =$

Property of the adj martices wrt dimension?

Example

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25

$$
\begin{aligned}
m[2,5] &= \min \begin{cases} m[2,2] + m[3,5] + p_1 p_2 p_5 &= 0 + 2500 + 35 \cdot 15 \cdot 20 &= 13{,}000\,, \\ m[2,3] + m[4,5] + p_1 p_3 p_5 &= 2625 + 1000 + 35 \cdot 5 \cdot 20 &= 7125\,, \\ m[2,4] + m[5,5] + p_1 p_4 p_5 &= 4375 + 0 + 35 \cdot 10 \cdot 20 &= 11{,}375 \end{cases} \\
&= 7125\,.
\end{aligned}
$$

Recursion?

A recursive algorithm

```
RECURSIVE-MATRIX-CHAIN(p, i, j)
1  if i == j
2      return 0
3  m[i, j] = ∞
4  for k = i to j − 1
5      q = RECURSIVE-MATRIX-CHAIN(p, i, k)
           + RECURSIVE-MATRIX-CHAIN(p, k + 1, j)
           + p_{i−1} p_k p_j
6      if q < m[i, j]
7          m[i, j] = q
8  return m[i, j]
```

Time Complexity

T(n) = #recursive calls invoked for the parameter n

T(n) =

Time Complexity

T(n) = #recursive calls invoked for the parameter n

$$T(n) = 1 + \sum_{k=1}^{n-1} T(k) + T(n-k)$$

Time Complexity

T(n) = #recursive calls invoked for the parameter n

$$T(n) = 1 + \sum_{k=1}^{n-1} T(k) + T(n-k)$$

$T(n)=\Omega(2^n)$

Efficient algorithm (DP)

m[i, j]=minimum number of scalar multiplications to compute A[i:j]

i=j? m[i,i]=

Efficient algorithm (DP)

m[i, j]=minimum number of scalar multiplications to compute A[i:j]

i=j? m[i,i]=0

i<j? m[i,j]=

Efficient algorithm (DP)

m[i, j]=minimum number of scalar multiplications to compute A[i:j]

i=j? $m[i,i]=0$

i<j? $m[i,j]=m[i,k] + m[k+1,j] + p_{i-1} p_k p_j$

- for some $i \le k < j$ (supposing the optimal split is at k)

Efficient algorithm (DP)

m[i, j]=minimum number of scalar multiplications to compute A[i:j]

i=j? $m[i,i]=0$

i<j? $m[i,j]=m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j$

- for some i≤k<j (supposing the optimal split is at k)

$$m[i,j] = \begin{cases} 0 & \text{if } i = j\ , \\ \min\{m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j : i \le k < j\} & \text{if } i < j\ . \end{cases}$$

Top-down approach

```
MEMOIZED-MATRIX-CHAIN(p)
1  n = p.length − 1
2  let m[1..n, 1..n] be a new table
3  for i = 1 to n
4       for j = i to n
5            m[i, j] = ∞
6  return LOOKUP-CHAIN(m, p, 1, n)
```

Min/Max?

Top-down approach

```
MEMOIZED-MATRIX-CHAIN(p)
1  n = p.length − 1
2  let m[1..n, 1..n] be a new table
3  for i = 1 to n
4      for j = i to n
5          m[i, j] = ∞
6  return LOOKUP-CHAIN(m, p, 1, n)
```

```
LOOKUP-CHAIN(m, p, i, j)
1  if m[i, j] < ∞
2      return m[i, j]
3  if i == j
4      m[i, j] = 0
5  else for k = i to j − 1
6          q = LOOKUP-CHAIN(m, p, i, k)
               + LOOKUP-CHAIN(m, p, k + 1, j) + p_{i−1} p_k p_j
7          if q < m[i, j]
8              m[i, j] = q
9  return m[i, j]
```

Bottom-up approach

Top-down approach

```
LOOKUP-CHAIN(m, p, i, j)
1  if m[i, j] < ∞
2        return m[i, j]
3  if i == j
4        m[i, j] = 0
5  else for k = i to j − 1
6              q = LOOKUP-CHAIN(m, p, i, k)
                   + LOOKUP-CHAIN(m, p, k + 1, j) + p_{i−1} p_k p_j
7              if q < m[i, j]
8                    m[i, j] = q
9  return m[i, j]
```

Bottom-up approach

#subproblems?

Each subproblem tries

O(n) split points.

```
MATRIX-CHAIN-ORDER(p, n)
  let m[1:n, 1:n] and s[1:n − 1, 2:n] be new tables
  for i = 1 to n                                  // chain length 1
      m[i, i] = 0
  for l = 2 to n                                  // l is the chain length
      for i = 1 to n − l + 1                      // chain begins at A_i
          j = i + l − 1                           // chain ends at A_j
          m[i, j] = ∞
          for k = i to j − 1
              q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
              if q < m[i, j]
                  m[i, j] = q                     // remember this cost
                  s[i, j] = k                     // remember this index
  return m and s
```

Solution

```
PRINT-OPTIMAL-PARENS(s, i, j)
  if i == j
       print "A"_i
```

Initial call is PRINT-OPTIMAL-PARENS$(s, 1, n)$

Solution

```
PRINT-OPTIMAL-PARENS(s, i, j)
  if i == j
        print "A"_i
  else print "("
        PRINT-OPTIMAL-PARENS(s, i, s[i, j])
        PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
        print ")"
```

Initial call is PRINT-OPTIMAL-PARENS$(s, 1, n)$

Running time

$\Theta(n^3)$

```
MATRIX-CHAIN-ORDER(p, n)
  let m[1:n, 1:n] and s[1:n − 1, 2:n] be new tables
  for i = 1 to n                                  // chain length 1
      m[i, i] = 0
  for l = 2 to n                                  // l is the chain length
      for i = 1 to n − l + 1                      // chain begins at A_i
          j = i + l − 1                           // chain ends at A_j
          m[i, j] = ∞
          for k = i to j − 1
              q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
              if q < m[i, j]
                  m[i, j] = q                     // remember this cost
                  s[i, j] = k                     // remember this index
  return m and s
```